Adaptive Adversarial Curriculum in Highway Environment

Edoardo Ensoli 1918623 & Daniel Munera Martinelli 2049054
Sapienza University of Rome

Table of contents

1 Project Introduction

2 Baselines

3 Curriculum Learning Approaches

4 Results

5 Considerations and Future Works

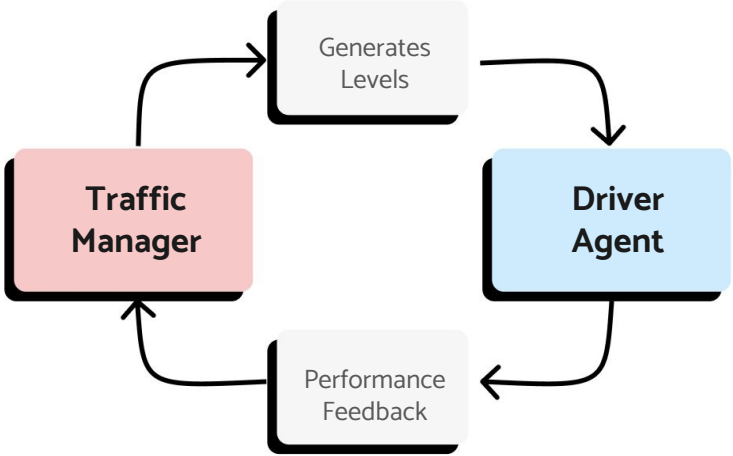
6 References

The Problem & Objective

Objective: Develop an advanced training framework for autonomous driving agents.

The Problem: Traditional Reinforcement Learning (RL) suffers from overfitting, poor generalization to ”edge cases,” and high computational inefficiency.

The Proposal: A dual-component system featuring a Student (Driver Agent) and an Instructor (Traffic Manager)



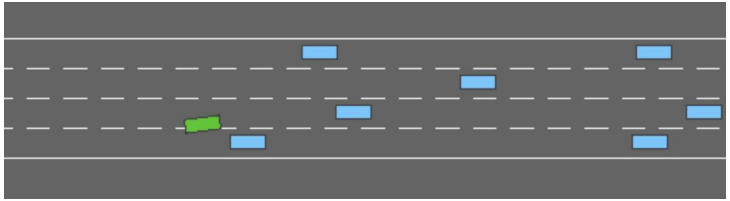
The Environment: Highway-Env

A **collection of environments for autonomous driving** and tactical decision-making tasks, developed and maintained by Edouard Leurent.

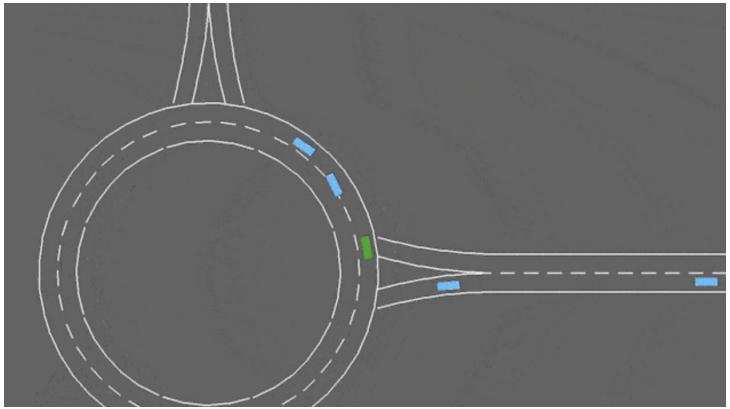
We worked mainly with **highway-v0** environment. **Adjustable parameters** to customize each level are:

- “lanes_count”
- “vehicles_count”
- “vehicles_density”
- “other_vehicles_type” (Politeness)

highway-v0



roundabout-v0



Highway-Env

The Environment: Highway-Env

Different options for observation space are available:

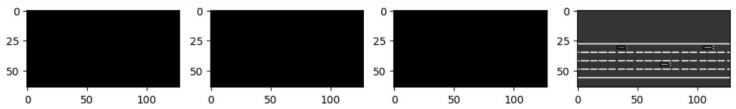
- **Kinematics**
- GreyscaleObservation
- OccupancyGrid
- Lidar

Action space could be continuous or discrete, where discrete one is a uniform quantization of the other:

```
ACTIONS_ALL = {
        0: 'LANE_LEFT',
        1: 'IDLE',
        2: 'LANE_RIGHT',
        3: 'FASTER',
        4: 'SLOWER'
    }
```

Highway-Env

GrayscaleObservation



Kinematics

Vehicle	x	y	v_x	v_y
ego-vehicle	5.0	4.0	15.0	0
vehicle 1	-10.0	4.0	12.0	0
vehicle 2	13.0	8.0	13.5	0
...	...	...	...	...
vehicle V	22.2	10.5	18.0	0.5

Table of contents

1 Project Introduction

2 Baselines

3 Curriculum Learning Approaches

4 Results

5 Considerations and Future Works

6 References

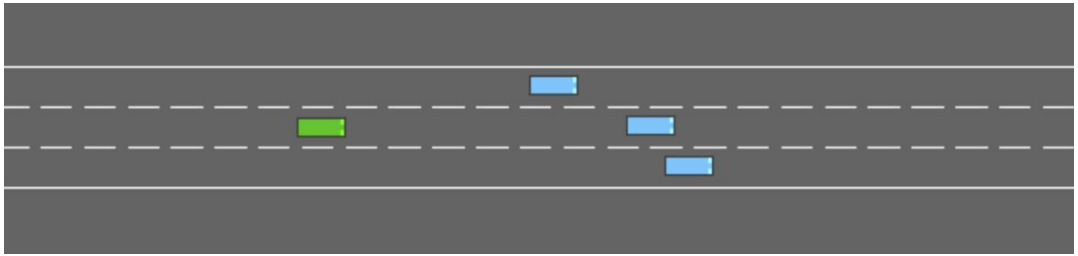
Training Baseline Models

We designed a level that neither too simple and nor too hard to train **2 baseline models**:

- **DQN**
- **PPO**

Baseline level includes:

- “lanes_count” = 3
- “vehicles_count” = 8
- “vehicles_density” = 0.8

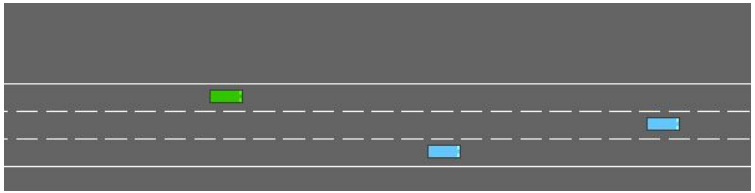


Baseline: Deep Q-Network (DQN)

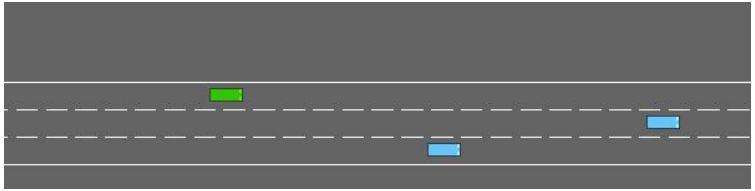
The DQN algorithm is a model-free, **off-policy** Reinforcement Learning method that **uses a deep neural network** to approximate the optimal action-value function (***Q****). Instead of maintaining a table for every possible state, it uses a network to predict the quality of each action given a state.

$$L(\theta) = \mathbb{E}\left[\left(\underbrace{r + \gamma \max_{a'} Q(s', a'; \theta^-)}_{\text{Target (Bellman)}} - \underbrace{Q(s, a; \theta)}_{\text{Prediction}}\right)^2\right]$$

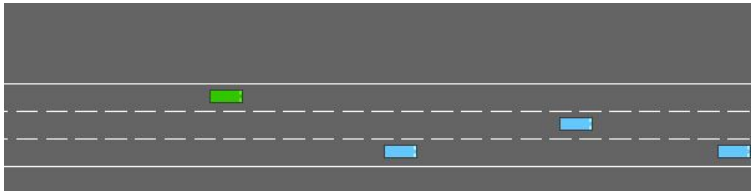
DQN after 500k env. steps of training



DQN after 1M env. steps of training



DQN after 1M env. steps of training (on **unseen level**)



Human-level control through deep reinforcement learning

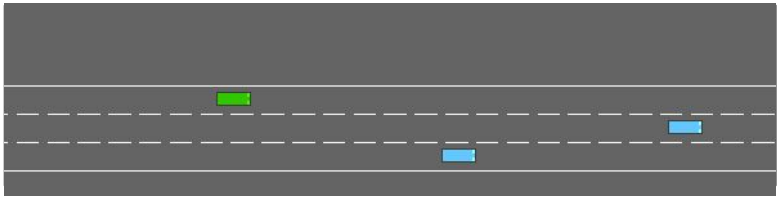
Baseline: Proximal Policy Algorithm (PPO)

PPO is a **policy-gradient** method that falls under the **Actor-Critic architecture**. Unlike DQN, which learns a value function (**Q**) and chooses actions based on those values, PPO directly optimizes the policy (**π**). Its defining feature is the use of a "trust region" logic that prevents the agent from making massive, destabilizing updates that could "collapse" the learning process.

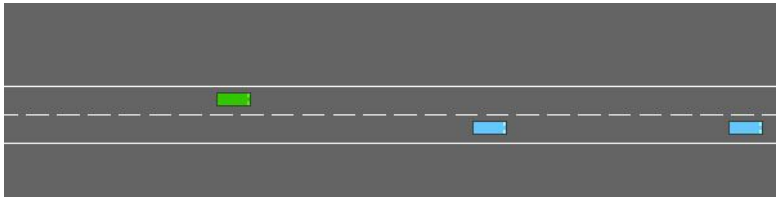
$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right]$$

PPO after 500k env. steps of training

PPO after 1M env. steps of training



PPO after 1M env. steps of training (on **unseen level**)



Proximal Policy Algorithm (PPO)

Table of contents

1 Project Introduction

2 Baselines

3 Curriculum Learning Approaches

4 Results

5 Considerations and Future Works

6 References

Curriculum Learning - CL

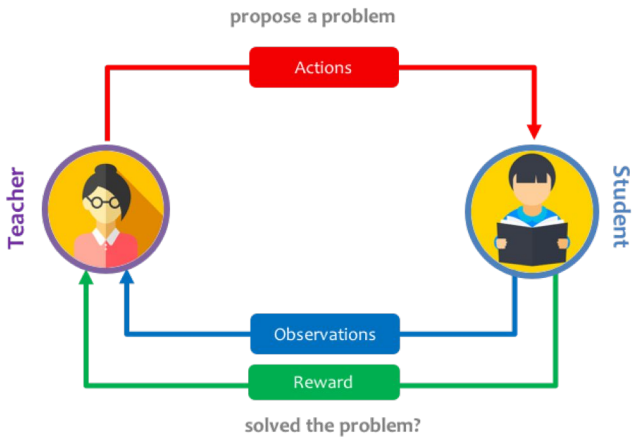
CL is a training strategy that mirrors human education by presenting simpler tasks first and **progressively increasing the difficulty** as the agent improves.

- **Avoiding Overfitting**: In standard RL, agents often ”memorize” a single fixed scenario. CL forces the agent to learn fundamental driving skills across a variety of easier levels before facing expert scenarios.
- **The Learning Frontier**: It focuses training on the ”Goldilocks zone”–scenarios that are neither too easy (wasting time) nor too hard (where the agent learns nothing).
- **Robustness**: By exposing the agent to a spectrum of behaviors, it develops a policy that generalizes better to ”edge cases” or unseen traffic conditions.

Curriculum Learning - CL

Two main methodologies are used to implement CL:

- **Manual Curricula** (hand-designed stages)
- **Automated Curriculum Learning** (algorithms that select or generate tasks dynamically)



Prioritized Level Replay (PLR)

PLR is an algorithm designed to improve generalization by focusing training on levels with highest **learning potential**.

Instead of training on random levels, PLR selectively replays levels where the agent performs poorly or experiences high uncertainty.

PLR estimates the agent's "**regret**" or learning potential on a specific level.

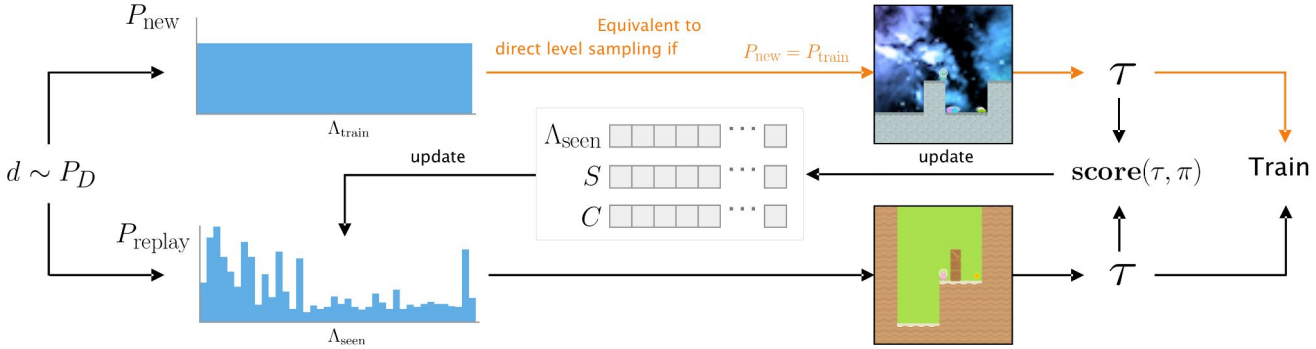
- **High Regret**: The agent failed or behaved sub-optimally, meaning there is still much to learn.
- **Low Regret:** The agent mastered the level; replaying it is a waste of computational resources.

Prioritized Level Replay

Prioritized Level Replay (PLR)

Our implementation of PLR tries to mimic with fidelity the one provided by **Facebook Research**, but we choose **DQN** from Stable Baselines 3 as learner algorithm instead of PPO .

The training follows this schema:

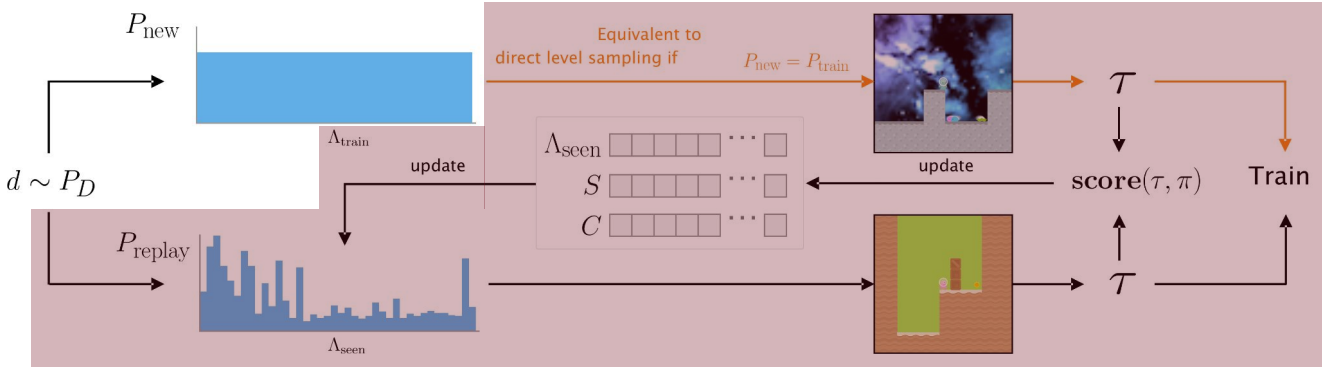


Prioritized Level Replay

Robust PLR

Robust PLR uses **2** distinct buffers, **Staging Buffer** filled during a Warm-Up phase and the **Working Buffer** used during Replay, which contains levels with assigned regret value.

During **Warm-Up** phase the agent acts randomly on randomly generated levels in order to fill the Staging Buffer (no training here).



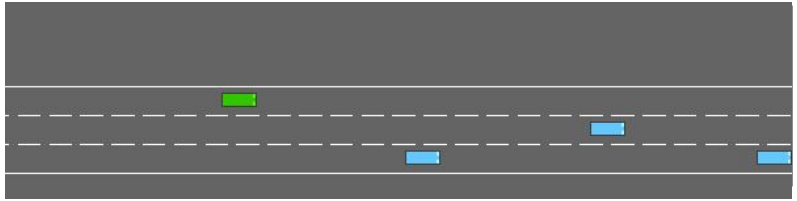
Prioritized Level Replay

Robust PLR

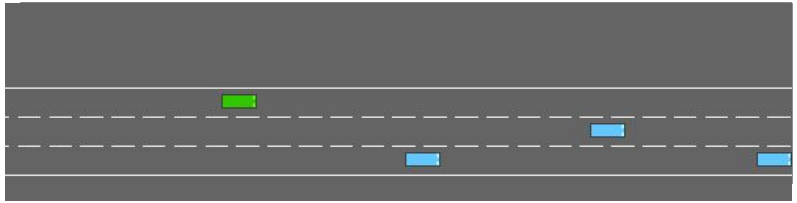
After the Staging Buffer reaches a capacity of **ρ** , Warm-Up phase ends and the agent start to update the policy on the levels provided by the curator and calculates regret of such levels:

- **Replay** with probability p = 95%: sample from Working Buffer (if empty, sample from Staging Buffer) and update Regret
- **Explore** with probability 1 - p = 5%: sample from Staging Buffer

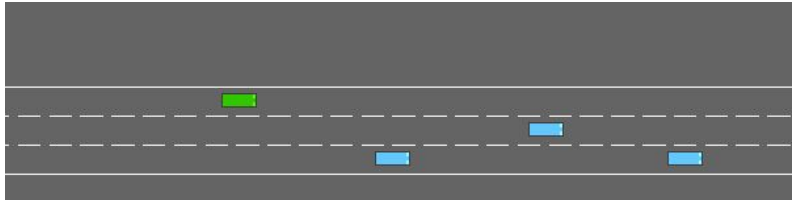
DQN+PLR (500K) on medium level



DQN+PLR (1M) on medium level



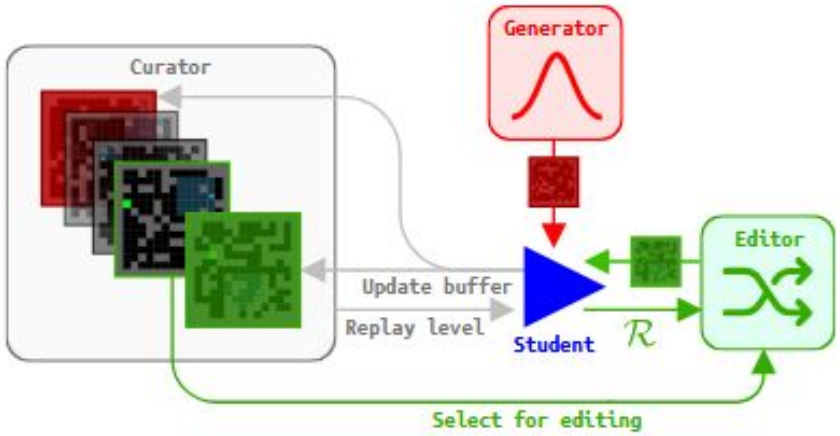
DQN+PLR (1M) on hard level



Prioritized Level Replay

Evolving Curricula with Regret-Based Environment Design (ACCEL)

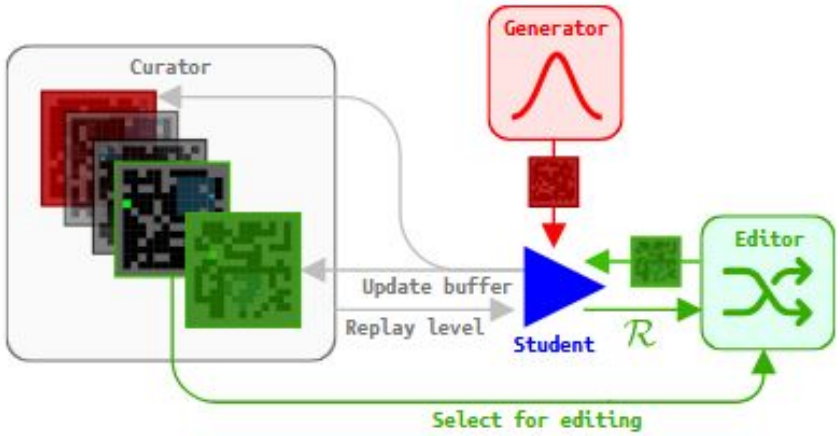
- **The Evolution of PLR:** While standard **PLR only replays** visited levels, ACCEL uses a **Level Mutation Editor** to actively generate new challenges.
- **High-Regret Mutations:** The system identifies levels with **high TD-error** (where the agent is most confused) and creates slightly modified variants of them.



ACCEL - Evolving Curricula with Regret-Based Environment Design

Evolving Curricula with Regret-Based Environment Design (ACCEL)

- **Parameter Perturbation:** It automatically **adjusts environmental factors** like vehicle density, lane counts, and spacing to find the ”hardest” version of a problem the agent can still learn from.
- **Infinite Curriculum:** This creates a **continuous stream** of relevant training data, preventing the agent from ever ”plateauing” on a fixed set of levels.



ACCEL - Evolving Curricula with Regret-Based Environment Design

Our 7-Stage Curriculum Design

We designed a structured progression where **each stage introduces** exactly one **new difficulty factor** (lanes, density, or duration) to ensure clear credit assignment.

The Difficulty Curve:

- **Stage 0-1:** Foundation (2-3 lanes, low density).
- **Stage 2-4:** Complexity (Increased traffic density and episode duration).
- **Stage 5-6:** Expert (Introduction of aggressive, unpredictable drivers and max lane count).

Fixed Constraints: To ensure the agent learns a robust policy, the **observation space** (kinematics) and **reward** signal (speed + collision) **remain constant** across all timesteps.

Our 7-Stage Curriculum Design

We implemented a **Competence Gate** to prevent the agent from "drowning" in difficult levels before learning the basics:

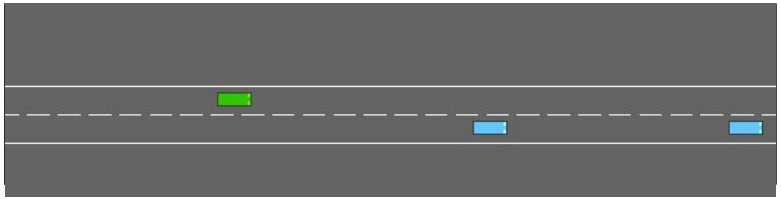
- **Foundation** (<50% survival): PLR/ACCEL are OFF; focus on basic driving.
- **The Ramp** (50-75% survival): Linear activation of level replay and mutations.
- **Full Throttle** (>75% survival): Full curriculum active (95% replay probability).

To advance to the next stage, the agent **must achieve an 80% survival rate** over a window of 50 "pure" evaluation episodes.

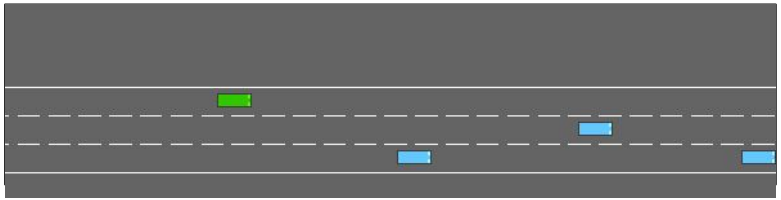
If progress plateaus for 200 episodes, the system automatically shifts focus to diagnostic evaluation to recover from the stall.

Our 7-Stage Curriculum Design

DQN+ACCEL (500K) on 2 lanes level



DQN+ACCEL (1M) on medium level



DQN+ACCEL (1M) on hard level

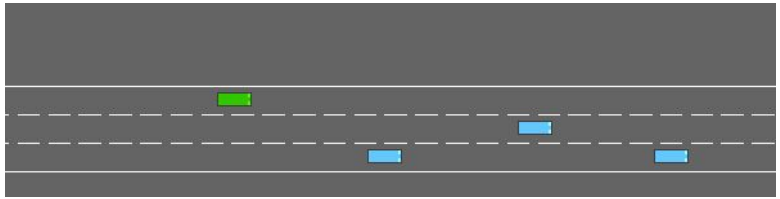


Table of contents

1 Project Introduction

2 Baselines

3 Curriculum Learning Approaches

4 Results

5 Considerations and Future Works

6 References

Performance Comparison 1 Million Steps

Easy

Model	Survival %	Avg Reward
dqn_accel	83.0	12.32
dqn_baseline	54.0	9.11
dqn_plr	57.0	-2.38
ppo	80.0	10.68

Baseline

Model	Survival %	Avg Reward
dqn_accel	87.0	20.62
dqn_baseline	82.0	20.11
dqn_plr	43.0	-1.06
ppo	92.0	15.1

Medium

Model	Survival %	Avg Reward
dqn_accel	67.0	16.01
dqn_baseline	64.0	15.29
dqn_plr	49.0	-2.32
ppo	77.0	8.33

Hard

Model	Survival %	Avg Reward
dqn_accel	31.0	9.06
dqn_baseline	26.0	7.26
dqn_plr	35.0	-4.54
ppo	19.0	-1.63

Performance Comparison 1 Million Steps

Easy

Model	Cars Overtaken	Distance (m)
dqn_accel	4.15	625.3
dqn_baseline	4.12	564.2
dqn_plr	0.31	386.4
ppo	3.81	588.5

Baseline

Model	Cars Overtaken	Distance (m)
dqn_accel	7.69	897.3
dqn_baseline	7.67	891.7
dqn_plr	1.15	498.9
ppo	5.17	819.7

Medium

Model	Cars Overtaken	Distance (m)
dqn_accel	9.01	798.4
dqn_baseline	8.77	772.5
dqn_plr	1.12	454.1
ppo	4.74	689.6

Hard

Model	Cars Overtaken	Distance (m)
dqn_accel	9.48	661.7
dqn_baseline	8.73	607.3
dqn_plr	1.47	434.0
ppo	3.91	438.5

Table of contents

1 Project Introduction

2 Baselines

3 Curriculum Learning Approaches

4 Results

5 Considerations and Future Works

6 References

Hardware & Training Constraints

Our work was harshly constrained by hardware limitations:

- Performing **~1 Million environment steps** during training requires **~2 hours**, depending on specific implementation. Many configurations of parameters needed to be tested in order to reach these results
- The target environment, like many other, is **CPU bounded**, making it difficult to speed up training on our consumer hardware
- Training an RL agent to SOTA performance requires resource on a whole other scale, e.g. authors of ACCEL state: "*We trained the ACCEL agent for **50,000 student PPO updates**, equivalent to approximately 3**B environment steps**–well under the approximately 500B environment steps used to train POET*"

Future Works

Future developments will focus on three key areas:

- **Extended Training:** Increasing the **number of iterations** to refine the agent's policy.
- **Hardware Optimization:** Developing a more efficient system to fully exploit **GPU acceleration** for faster training cycles or looking toward emerging "Distributed Reinforcement Learning" approaches
- **Robustness and Generalization:** Training the agent to maintain high performance in **varied and unseen** driving environments.

Table of contents

1 Project Introduction

2 Baselines

3 Curriculum Learning Approaches

4 Results

5 Considerations and Future Works

6 References

References

- **Papers:**
  - Human-level control through deep reinforcement learning (DQN)
  - Proximal Policy Algorithm (PPO)
  - Prioritized Level Replay
  - ACCEL - Evolving Curricula with Regret-Based Environment Design
- **Environment:**
  - Highway-Env
- **Slides:**
  - https://github.com/pietro-nardelli/sapienza-ppt-template

    [Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)]
- **Github**
  - https://github.com/edoardoensoli/AAC-Highway

Thank you for the attention!

Edoardo Ensoli 1918623 & Daniel Munera Martinelli 2049054
Sapienza University of Rome